



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

Wydział Informatyki
Środowiska Udostępniania Usług
Grupa 4 - czwartek 13:15

MCP-PK

*Obserwowalność Kubernetes z Prometheusem kontrolowanym
przez serwer MCP*

*Kubernetes Observability with Prometheus controlled by MCP
server*

Autorzy: Dominik Drózdź, Szymon Miękina, Magdalena Pabisz, Marcin Walendzik

Kraków, 2026

Spis treści

1	Wprowadzenie	4
2	Podstawy teoretyczne i stos technologiczny	5
2.1	Kubernetes	5
2.2	Prometheus	5
2.3	MCP Server	6
2.4	Large Language Model	6
2.5	Grafana	6
3	Opis studium przypadku	7
3.1	Aplikacja testowa: Microservices-demo	7
3.1.1	Architektura i zasada działania	7
3.1.2	Uzasadnienie wyboru	8
3.2	Opis przypadku użycia	8
4	Wysokopoziomowy opis architektury	9
5	Szczegółowy opis architektury	10
5.1	Warstwa orkiestracji – Kubernetes	10
5.2	Warstwa aplikacyjna – Online Boutique	11
5.3	Warstwa monitorowania – Prometheus	11
5.4	Warstwa wizualizacji – Grafana	12
5.5	Warstwa inteligentnej konfiguracji – Large Language Model	13
5.6	Warstwa pośrednicząca – MCP Server	13
6	Konfiguracja środowiska	15
6.1	Utworzenie klastra EKS	15
6.2	Konfiguracja dostępu do klastra	15
6.3	Weryfikacja konfiguracji środowiska	15
6.4	Konfiguracja przestrzeni nazw (namespace)	16
7	Sposób instalacji	17
7.1	Uruchomienie aplikacji	17
7.2	Monitoring i wizualizacja	17
7.2.1	Prometheus UI	18
7.2.2	Grafana	18
7.3	Serwer MCP	19
7.4	Large Language Model	22

8	Wdrożenie wersji demonstracyjnej	23
8.1	Integracja serwera MCP z LLMem	23
8.2	Zmiana interwału pobierania metryk	24
8.3	Dodanie nowej reguły alertowania	24
9	Opis demonstracji	25
9.1	Zmiana interwału zbierania metryk	25
9.2	Stworzenie reguły alertowania	26
10	Podsumowanie i wnioski	29

Rozdział 1

Wprowadzenie

Obserwowalność (observability) jest kluczowym elementem nowoczesnych systemów cloud-native. Umożliwia pomiar, analizę oraz zrozumienie wewnętrznego stanu systemu na podstawie jego zewnętrznych wyników. Opiera się na trzech głównych typach danych: metrykach, które pozwalają monitorować ogólną efektywność systemu, logach, które opisują zdarzenia zachodzące w systemie, oraz śladach, które umożliwiają śledzenie przepływu operacji.

Współczesne aplikacje coraz częściej składają się z niezależnych mikroserwisów i działają w środowiskach kontenerowych, takich jak Kubernetes. W systemach rozproszonych niemożliwe jest ręczne śledzenie zależności pomiędzy usługami i przewidywanie potencjalnych awarii. Z tego względu niezbędne jest zastosowanie narzędzi wspierających obserwowalność. Kubernetes nie posiada wbudowanych mechanizmów monitorowania, dlatego integruje się go z zewnętrznymi narzędziami, takimi jak Prometheus. Dzięki obserwowalności można zrozumieć zależności pomiędzy poszczególnymi usługami, zlokalizować nieefektywne fragmenty kodu czy wolne zapytania do bazy danych. W konsekwencji odpowiednie monitorowanie pozwala zapewnić niezawodny, wydajny i skalowalny system.

Celem projektu jest przygotowanie rozwiązania do obserwowalności aplikacji uruchomionej w środowisku Kubernetes z wykorzystaniem Prometheusa. Dodatkowo infrastruktura odpowiedzialna za obserwowalność jest rozszerzona o serwer MCP (Model Context Protocol), który pozwala na wykorzystanie dużego modelu językowego. Dzięki temu możliwe jest zarządzanie konfiguracją Prometheusa za pomocą poleceń w języku naturalnym. Serwer MCP pełni rolę warstwy pośredniczącej, która przekształca język naturalny na operacje konfiguracyjne wykonywane w Prometheusie. Takie podejście pozwala częściowo zautomatyzować zarządzanie infrastrukturą monitorującą oraz zwiększyć elastyczność i wygodę jej obsługi.

Rozdział 2

Podstawy teoretyczne i stos technologiczny

Wybór technologii do tworzenia projektu został częściowo podyktowany sugestiami zawartymi w dokumentach z zajęć laboratoryjnych przedmiotu Środowiska Udostępniania Usług, jak i również powszechnością konkretnych rozwiązań w przypadku danej funkcjonalności. Większość z poniżej opisanych narzędzi jest rozwijana jako technologie typu open-source, co zapewnia często większą swobodę przy korzystaniu z produktu.

2.1. Kubernetes

Kubernetes jest narzędziem stworzonym do orkiestracji kontenerów. Stanowi obecnie najpowszechniejszą technologię z tego zakresu. Jej produkcja została rozpoczęta przez Google, a obecnie jest rozwijana jako produkt otwartoźródłowy. Kubernetes opiera swoje działanie o kontrolę nad aplikacjami zamkniętymi w kontenerach. Technologia ta pozwala na dynamiczne zarządzanie usługami zapewniając łatwą skalowalność, kluczową obecnie w wielkich środowiskach usługowych. System dostosowuje automatycznie ilość potrzebnych instancji kontenerów w oparciu o zdefiniowane przez deweloperów reguły oraz obciążenie. W kontekście naszego projektu, warto wspomnieć również, że Kubernetes wystawia endpointy, które służą warstwie observability do czytania danych metrycznych pomagających określić stan serwisów oraz zasobów sprzętowych.

2.2. Prometheus

Prometheus to technologia open-source służąca do monitorowania stanów zasobów w złożonych aplikacjach i systemach o wielu usługach. Zarządza on danymi wystawionymi przez aplikację, agregując je w bazie danych typu time-series database. W stosunku do aplikacji działa jako klient, który aktywnie pobiera wyznaczone dane dostępne na endpointach HTTP. Skupia się on na danych numerycznych z zakresu m.in. zużycia zasobów pamięci operacyjnej oraz masowej, ilości wystąpień wyjątków, czasu odpowiedzi aplikacji na zapytania. Dane te są dostępne następnie za pomocą wbudowanego języka zapytań PromQL. Narzędzie to udostępnia również mechanizm powiadamiania wskazanych użytkowników nt. awarii, bądź nieefektywnego zużycia poszczególnych zasobów. W kontekście wielousługowych aplikacji Prometheus jest obecnie pomocnym i skutecznym narzędziem pozwalającym na ciągłe monitorowanie stanu aplikacji, dostosowywanie zasobów pod wskazany cel, będąc jednocześnie dostępnym w formie wolnego oprogramowania.

2.3. MCP Server

MCP Server jest warstwą stanowiącą pomost między instrukcją w języku naturalnym a konfiguracją w systemie. Działa w oparciu o otwartoźródłowy protokół internetowy MCP (*Model Context Protocol*). Udostępnia on warstwę komunikacyjną dla modelu, dzięki której może on dynamicznie korzystać z zasobów i funkcji serwera.

2.4. Large Language Model

Duży model językowy jest elementem najistotniejszym z perspektywy instrukcji wejściowych. Od strony technicznej jest to specjalnie wytrenowana sieć neuronowa zdolna do generowania tekstu. W przypadku opisywanego projektu powinna stanowić pośrednika, który będzie umiał przetłumaczyć instrukcje użytkownika w języku naturalnym na odpowiednie zapytania techniczne i operacje konfiguracyjne. Jego wyjście kierowane do serwera MCP sprawi, że użytkownik będzie mógł aktywnie konfigurować element observability z pozycji instrukcji sformułowanych "po ludzku".

2.5. Grafana

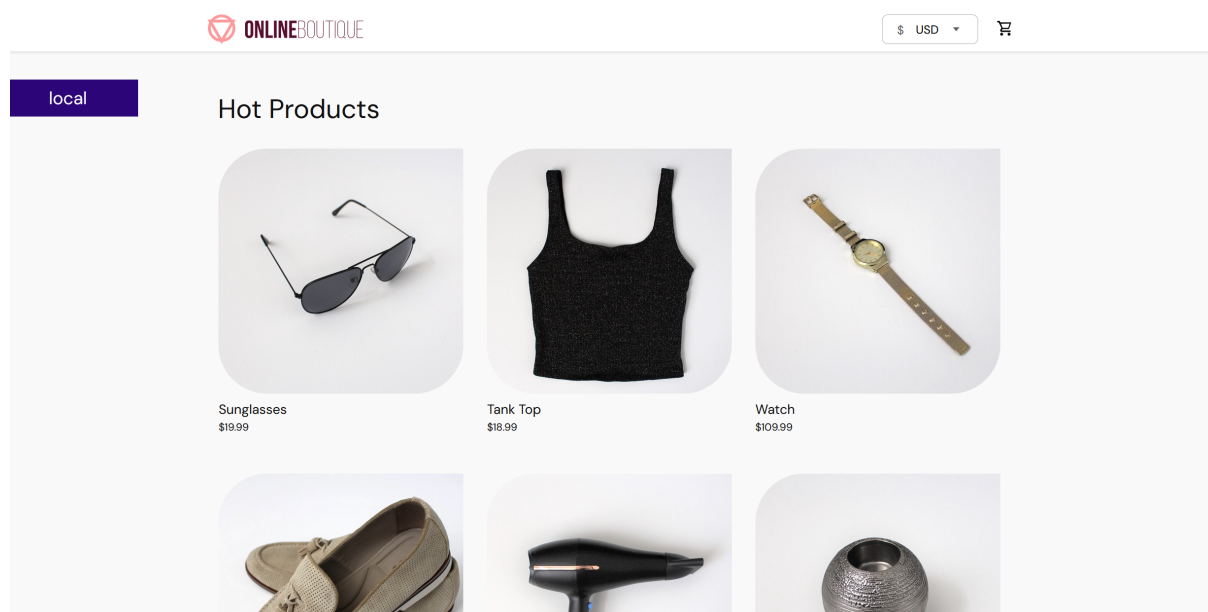
Grafana jest elementem służącym do wizualizacji danych przechowywanych w bazie danych Prometheusa. Jej przewagą jest większa możliwość w kwestii wizualizacji. Dzięki niej można utrzymywać kompozycje różnych wykresów zależnie od celu. Grafana, podobnie jak poprzednie technologie, jest również dostępna jako technologia open-source.

Rozdział 3

Opis studium przypadku

3.1. Aplikacja testowa: Microservices-demo

W projekcie jako obiekt monitorowania wykorzystano aplikację **Google Microservices Demo** (znaną również pod nazwą *Online Boutique*). Jest to otwartoźródłowa aplikacja demonstracyjna, która symuluje działanie sklepu internetowego. Składa się ona z 11 mikroservisów napisanych w różnych językach programowania (m.in. Go, Python, Java, C#, Node.js), które komunikują się ze sobą za pomocą protokołu gRPC.



Rysunek 3.1: Aplikacja Online Boutique

3.1.1. Architektura i zasada działania

Aplikacja została zaprojektowana w sposób natywny dla chmury (*cloud-native*), co oznacza, że każdy jej komponent pełni ściśle określoną rolę:

- **Frontend:** Serwer WWW napisany w Go, który serwuje interfejs użytkownika.
- **Product Catalog Service:** Dostarcza informacje o produktach z plików JSON.

- **Cart Service:** Zarządza koszykami użytkowników, wykorzystując bazę danych Redis do przechowywania danych tymczasowych.
- **Currency Service:** Odpowiada za przeliczanie walut.
- **Payment Service:** Obsługuje procesy płatności (symulacja).
- **Shipping Service:** Oblicza koszty dostawy.
- **Ad Service:** Serwuje reklamy kontekstowe.

Każdy z serwisów jest odizolowanym kontenerem, co pozwala na niezależne skalowanie komponentów wewnątrz klastra Kubernetes.

3.1.2. Uzasadnienie wyboru

Wybór tej konkretnej aplikacji do projektu obserwowalności z wykorzystaniem serwera MCP i Prometheusa jest podyktowany kilkoma kluczowymi czynnikami:

1. **Różnorodność technologiczna:** Dzięki zastosowaniu wielu języków i frameworków, aplikacja stanowi idealne środowisko do testowania uniwersalności narzędzi monitorujących.
2. **Złożoność ruchu sieciowego:** Komunikacja gRPC pomiędzy wieloma ogniwami systemu pozwala na generowanie realistycznych metryk dotyczących opóźnień (latency), liczby zapytań oraz błędów, co jest kluczowe dla demonstracji możliwości Prometheusa.
3. **Gotowość do wdrożenia na Kubernetes:** Aplikacja posiada natywne manifesty YAML oraz wsparcie dla Helm, co znacząco ułatwia jej szybką orkiestrację i integrację z infrastrukturą monitorującą.
4. **Realizm przypadku użycia:** Symulacja sklepu internetowego pozwala na łatwe definiowanie reguł biznesowych i alertów (np. powiadomienie o błędzie w procesie płatności lub zbyt wolnym ładowaniu katalogu produktów), które mogą być następnie modyfikowane za pomocą poleceń w języku naturalnym przez serwer MCP.

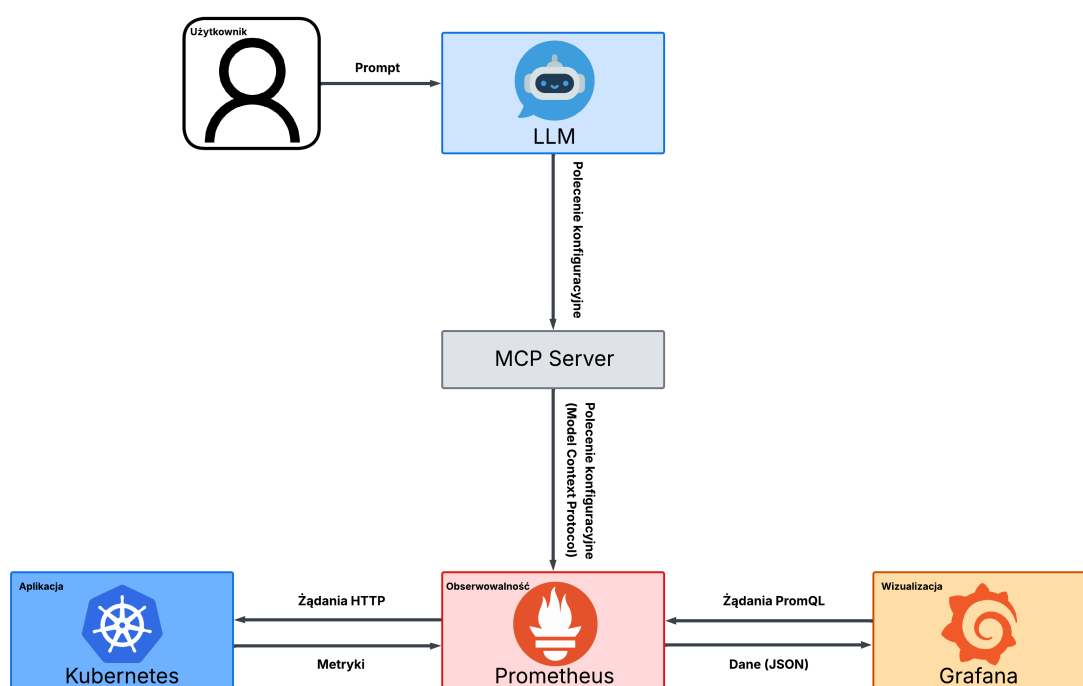
3.2. Opis przypadku użycia

Poniżej opisano kolejne operacje, które zostaną przeprowadzone w celu demonstracji przypadku użycia.

1. Uruchomienie klastra **Kubernetesa** z wybraną aplikacją oraz wdrożenie narzędzia monitorującego **Prometheus** wraz z podstawową konfiguracją zbierającą dane dotyczące zużycia CPU, a także integracja **serwera MCP** oraz systemu wizualizacji **Grafana** z Prometheusem.
2. Wydanie poleceń w języku naturalnym przy użyciu dużego modelu językowego, w wyniku których dokonane zostaną następujące zmiany w konfiguracji Prometheusa:
 - zmniejszenie interwału zbierania metryk,
 - dodanie reguły alertowania o wysokim zużyciu procesora,
3. Zaobserwowanie wprowadzonych zmian w konfiguracji.

Rozdział 4

Wysokopoziomowy opis architektury



Rysunek 4.1: Architektura wysokopoziomowa

Rozdział 5

Szczegółowy opis architektury

Architektura projektu została zaprojektowana zgodnie z podejściem *cloud-native* oraz modelem mikroserwisowym. Całość rozwiązania działa w środowisku Kubernetes i składa się z kilku współpracujących warstw odpowiedzialnych za uruchamianie aplikacji, zbieranie metryk, analizę danych, wizualizację oraz komunikację z dużym modelem językowym poprzez protokół MCP.

5.1. Warstwa orkiestracji – Kubernetes

Kubernetes pełni rolę centralnej platformy uruchomieniowej dla wszystkich komponentów systemu. Odpowiada za:

- uruchamianie oraz zarządzanie kontenerami aplikacji,
- automatyczne restartowanie usług w przypadku awarii,
- skalowanie mikroservisów w zależności od obciążenia,
- zarządzanie komunikacją sieciową pomiędzy usługami,
- utrzymywanie deklaratywnego stanu infrastruktury,
- udostępnianie endpointów metrycznych dla warstwy observability.

Każdy komponent projektu działa jako osobny Pod. Kubernetes zarządza ich cyklem życia oraz umożliwia niezależne skalowanie poszczególnych usług. Dzięki temu środowisko jest odporne na awarie i może dynamicznie reagować na zmiany obciążenia.

W projekcie Kubernetes hostuje:

- aplikację mikroservisową Online Boutique,
- Prometheusa,
- Grafanę,
- serwer MCP,
- dodatkowe komponenty monitorujące.

5.2. Warstwa aplikacyjna – Online Boutique

Aplikacja demonstracyjna Google Microservices Demo stanowi monitorowany system wykorzystywany w projekcie. Szczegółowy opis komponentów aplikacji oraz ich funkcjonalności został przedstawiony w podrozdziale **3.1.1 Architektura i zasada działania**.

Każdy mikroserwis aplikacji działa jako niezależny kontener uruchamiany w środowisku Kubernetes. Komunikacja pomiędzy usługami odbywa się głównie z wykorzystaniem protokołu gRPC, co pozwala generować realistyczny ruch sieciowy oraz dane telemetryczne wykorzystywane przez warstwę observability.

Aplikacja pełni rolę źródła metryk monitorowanych przez Prometheusa. Poszczególne komponenty generują dane dotyczące między innymi:

- czasu odpowiedzi usług,
- liczby żądań,
- liczby błędów,
- wykorzystania procesora,
- zużycia pamięci,
- komunikacji sieciowej pomiędzy mikroserwisami.

Dzięki architekturze mikroserwisowej możliwe jest niezależne monitorowanie każdego komponentu systemu oraz analiza wpływu poszczególnych usług na działanie całej aplikacji.

5.3. Warstwa monitorowania – Prometheus

Prometheus pełni rolę głównego systemu monitorowania i agregacji metryk. Jest odpowiedzialny za:

- cykliczne pobieranie danych metrycznych z endpointów HTTP,
- przechowywanie danych w bazie typu *time-series database*,
- analizę danych czasowych,
- wykonywanie zapytań PromQL,
- obsługę reguł alertowania,
- monitorowanie klastra Kubernetes oraz aplikacji.

Prometheus działa w modelu *pull-based*, co oznacza, że samodzielnie odpytuje monitorowane usługi o aktualne metryki. W projekcie pobiera dane m.in. z:

- kubeletów klastra Kubernetes,
- node-exporterów,
- komponentów systemowych Kubernetes,

- mikroserwisów aplikacji,
- komponentów infrastrukturalnych.

Prometheus przechowuje dane historyczne dotyczące działania systemu, co pozwala analizować zmiany wydajności w czasie oraz wykrywać anomalie.

Istotnym elementem jest również język PromQL, umożliwiający:

- agregację danych,
- filtrowanie metryk,
- obliczanie średnich i percentyli,
- analizę trendów,
- definiowanie warunków alertowych.

5.4. Warstwa wizualizacji – Grafana

Grafana odpowiada za prezentację danych zbieranych przez Prometheusa w formie czytelnych dashboardów i wykresów.

Do głównych zadań Grafany należą:

- wizualizacja metryk systemowych i aplikacyjnych,
- tworzenie interaktywnych dashboardów,
- analiza zmian metryk w czasie rzeczywistym,
- prezentowanie alertów i stanów systemu,
- agregowanie danych z wielu źródeł.

Grafana komunikuje się bezpośrednio z Prometheusem wykorzystując zapytania HTTP, w których przekazuje zapytanie PromQL. Dzięki temu możliwe jest dynamiczne generowanie wykresów przedstawiających:

- wykorzystanie CPU,
- zużycie pamięci,
- opóźnienia odpowiedzi usług,
- liczbę błędów,
- ruch sieciowy,
- stan kontenerów oraz podów Kubernetes.

Dashboards Grafany umożliwiają administratorowi szybkie wykrywanie problemów wydajnościowych oraz analizę zachowania systemu pod obciążeniem.

5.5. Warstwa inteligentnej konfiguracji – Large Language Model

Duży model językowy (LLM) stanowi interfejs komunikacji użytkownika z systemem monitorowania. Jego zadaniem jest interpretacja poleceń wydawanych w języku naturalnym.

Model językowy odpowiada za:

- rozpoznawanie intencji administracyjnych,
- generowanie odpowiednich operacji konfiguracyjnych,
- tłumaczenie języka naturalnego na operacje techniczne,
- wspieranie automatyzacji konfiguracji observability.

Przykładowe polecenia użytkownika zgodne z wymaganiami projektu:

- „zmniejsz interwał zbierania metryk do 5 sekund”,
- „dodaj alert dla wysokiego zużycia pamięci”,
- „monitoruj metrykę zużycia CPU”.

Model językowy nie komunikuje się bezpośrednio z Prometheusem, lecz korzysta z warstwy pośredniczącej w postaci serwera MCP.

5.6. Warstwa pośrednicząca – MCP Server

Serwer MCP (*Model Context Protocol Server*) pełni rolę pośrednika pomiędzy dużym modelem językowym a infrastrukturą monitorującą.

Jego główne zadania obejmują:

- udostępnianie zunifikowanego interfejsu komunikacyjnego dla modelu językowego,
- interpretację żądań generowanych przez LLM,
- wykonywanie operacji konfiguracyjnych,
- komunikację z API Prometheusa,
- kontrolę dostępu do infrastruktury monitorującej.

Serwer MCP działa jako warstwa abstrakcji pomiędzy językiem naturalnym a konfiguracją techniczną. Dzięki temu model językowy nie musi znać szczegółów implementacyjnych Prometheusa.

Przykładowy przepływ działania:

1. użytkownik wydaje polecenie w języku naturalnym,
2. LLM interpretuje polecenie,
3. serwer MCP otrzymuje ustrukturyzowane żądanie,

4. MCP interpretuje żądanie i wykonuje odpowiednią operację na Prometheusie,
5. zmieniona konfiguracja wpływa na działanie monitoringu,
6. rezultaty są widoczne w Grafanie.

Takie podejście pozwala znacząco uprościć zarządzanie konfiguracją observability oraz ograniczyć konieczność ręcznej edycji manifestów konfiguracyjnych środowiska.

Rozdział 6

Konfiguracja środowiska

Projekt został wykonany w oparciu o chmurę publiczną **AWS** oraz udostępniony w ramach zajęć kurs AWS Academy Learner Lab. Środowisko zostało zbudowane z wykorzystaniem usługi **Amazon Elastic Kubernetes Service (EKS)**. W ramach konfiguracji utworzono dedykowany klaster Kubernetes przeznaczony do realizacji projektu.

Wszystkie operacje związane z konfiguracją środowiska oraz zarządzaniem klastrem Kubernetes zostały przeprowadzone w środowisku **Windows Subsystem for Linux (WSL)**, zapewniającym kompatybilność z narzędziami CLI systemu Linux.

6.1. Utworzenie klastra EKS

W pierwszym kroku utworzono klaster Kubernetes o nazwie `suu-project`. Klaster został skonfigurowany w regionie AWS `us-east-1`. W ramach infrastruktury obliczeniowej dodano grupę węzłów (node group) składającą się z 5 instancji EC2 typu `t3.large`.

6.2. Konfiguracja dostępu do klastra

W celu umożliwienia komunikacji z klastrem skonfigurowano lokalne środowisko użytkownika poprzez plik:

```
~/.aws/credentials
```

zawierający dane uwierzytelniające AWS CLI. Następnie, za pomocą AWS CLI, zweryfikowano status klastra:

```
aws eks describe-cluster --region us-east-1 --name suu-project --query cluster.status
```

Skonfigurowano również dedykowane narzędzie `kubectl` do komunikacji z klastrem EKS:

```
aws eks --region us-east-1 update-kubeconfig --name suu-project
```

6.3. Weryfikacja konfiguracji środowiska

Po wykonaniu powyższych kroków, zweryfikowano poprawność konfiguracji środowiska Kubernetes. Dostępność węzłów klastra sprawdzono za pomocą polecenia:

```
kubectl get nodes
```

Dodatkowo zweryfikowano stan komponentów systemowych Kubernetesa działających w namespace kube-system:

```
kubectl --namespace kube-system get pods
```

6.4. Konfiguracja przestrzeni nazw (namespace)

W celu logicznego podziału zasobów w klastrze Kubernetes utworzono kilka przestrzeni nazw (namespace), które odpowiadają różnym komponentom systemu:

- monitoring - przeznaczony dla komponentów monitoringu, takich jak Prometheus Operator oraz Grafana
- boutique-app - przestrzeń nazw dla głównej aplikacji
- mcp-demo - przestrzeń nazw dla serwera MCP oraz modeli LLM

Definicja namespace'ów znajduje się w pliku src/namespaces.yaml. Utworzenie przestrzeni nazw odbywa się za pomocą komendy:

```
kubectl apply -f src/namespaces.yaml
```


Rozdział 7

Sposób instalacji

W celu stworzenia założonego projektu dołączono kilka narzędzi zajmujących się jego wyszczególnionymi częściami. Wszystkie użyte narzędzia są darmowe, a zdecydowana większość z nich znajduje się aktualnie pod opieką CNCF (*ang. Cloud Native Computing Foundation*).

7.1. Uruchomienie aplikacji

Deployment aplikacji testowej Google Microservices Demo (Online Boutique) przeprowadzono za pomocą polecenia:

```
kubectl apply -n boutique-app -f \
https://raw.githubusercontent.com/GoogleCloudPlatform/microservices-demo/release/v0.10.1/
release/kubernetes-manifests.yaml
```

Powyższa komenda inicjuje utworzenie wszystkich zasobów Kubernetes na podstawie manifestu udostępnionego w repozytorium GitHub projektu Google Microservices Demo.

W celu weryfikacji poprawności deploymentu oraz uzyskania dostępu do interfejsu webowego aplikacji wykonano przekierowanie portów (port-forwarding) do usługi frontendu:

```
kubectl port-forward -n boutique-app svc/frontend 8088:80
```

Po uruchomieniu przekierowania aplikacja była dostępna w przeglądarce pod adresem <http://localhost:8088>.

7.2. Monitoring i wizualizacja

W celu wdrożenia systemu monitoringu wykorzystano narzędzie **Helm**, które służy do zarządzania pakietami aplikacji w Kubernetes. Helm umożliwia automatyczne instalowanie, aktualizowanie oraz konfigurowanie złożonych aplikacji na podstawie gotowych wykresów (charts).

W ramach konfiguracji dodano repozytorium Helm oraz zainstalowano stos kube-prometheus-stack, który zawiera Prometheus Operator oraz Grafanę.

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts

helm repo update
```

```
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --set grafana.enabled=true
```

Weryfikacja poprawności przeprowadzenia tego kroku może polegać na sprawdzeniu stanu podów oraz usług (serwisów) znajdujących się w przestrzeni nazw monitoring:

```
kubectl -n monitoring get pods
kubectl -n monitoring get svc
```

7.2.1. Prometheus UI

W celu uzyskania dostępu do interfejsu użytkownika Prometheusa wykonano przekierowanie portów (port-forwarding), umożliwiające lokalny dostęp do usługi działającej w klastrze Kubernetes.

```
kubectl -n monitoring port-forward svc/monitoring-kube-prometheus-prometheus 9090:9090
```

Po uruchomieniu przekierowania możliwe jest otwarcie interfejsu Prometheusa w przeglądarce pod adresem `http://localhost:9090`.

W sekcji `/targets` interfejsu Prometheusa widoczne są tzw. **ServiceMonitory**, które definiują zestawy usług oraz endpointów podlegających monitoringowi. Określają one, które komponenty klastra Kubernetes są scrapowane przez Prometheusa oraz z jaką częstotliwością odbywa się zbieranie metryk.

7.2.2. Grafana

Dostęp do interfejsu użytkownika Grafany został uzyskany poprzez przekierowanie portów (port-forwarding), co umożliwia lokalne połączenie z usługą działającą w klastrze Kubernetes:

```
kubectl port-forward svc/monitoring-grafana 3000:80 -n monitoring
```

Po uruchomieniu przekierowania interfejs Grafany dostępny jest w przeglądarce pod adresem `http://localhost:3000`.

Domyślne dane logowania do systemu Grafana:

- **login:** admin
- **hasło** można uzyskać za pomocą polecenia

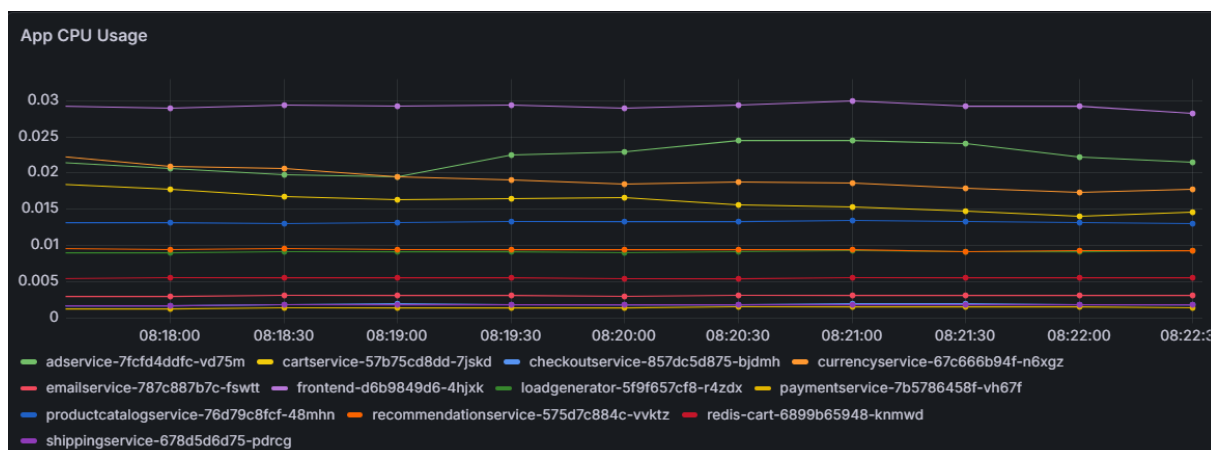
```
kubectl --namespace monitoring get secrets monitoring-grafana -o jsonpath="{.data.admin-  
password}" | base64 -d ; echo
```

Wizualizacja przykładowej metryki

W celu weryfikacji poprawności konfiguracji środowiska monitoringu w interfejsie Grafany wygenerowano wykres przedstawiający średnie zużycie CPU przez pody w przestrzeni nazw `boutique-app`. Do wizualizacji danych wykorzystano następujące zapytanie:

```
1 sum(rate(container_cpu_usage_seconds_total{namespace="boutique-app"}[5m]))
   by (pod)
```

Otrzymany wykres został przedstawiony na Rysunku 7.1.



Rysunek 7.1: Średnie zużycie CPU przez pody w namespace boutique-app

7.3. Serwer MCP

Wzorując się na przykładzie serwera MCP zaprezentowanego podczas laboratorium, przygotowano własną implementację serwera MCP umożliwiającego zmianę konfiguracji Prometheus w klastrze Kubernetes. Implementacja wykorzystuje pomocniczą funkcję `run_kubectl` oraz metody oznaczone dekoratorem `@mcp.tool`, które mogą być wywoływane przez agenta AI.

Poniżej przedstawiono funkcję umożliwiającą zmianę interwału zbierania metryk dla wybranego ServiceMonitora w klastrze:

```
1 @mcp.tool()
2 def update_servicemonitor_scrape_interval(
3     name: str,
4     interval: str,
5     namespace: str = "monitoring"
6 ) -> dict:
7     """
8     Update scrape interval on ServiceMonitor.
9
10    The interval MUST be a valid Prometheus duration string.
11
12    Valid formats:
13    - 15s
14    - 30s
15    - 1m
16    - 5m
17
18    Invalid:
19    - 15
20    - 15 seconds
21    - "one minute"
```

```

22
23     No automatic conversion is performed unless explicitly handled in code.
24     """
25
26     patch = [
27         {
28             "op": "replace",
29             "path": "/spec/endpoints/0/interval",
30             "value": interval
31         }
32     ]
33
34     result = run_kubectl([
35         "-n", namespace,
36         "patch", "servicemonitor", name,
37         "--type=json",
38         "-p", json.dumps(patch)
39     ])
40
41     return {
42         "success": result.get("returncode", 1) == 0,
43         "status": "command executed successfully" if
44         result.get("returncode", 1) == 0 else "command failed",
45         "error": result.get("stderr") if result.get("returncode", 1) != 0
46     else None
47     }

```

Serwer jest wyposażony również w funkcjonalność tworzenia reguł alarmowych w kontekście zużycia zasobów procesora przez aplikację. Narzędzie odpowiadające za to pozwala na to poprzez nadanie żądanego progu powyżej, którego wnoszony jest alarm. W tej funkcjonalności można również zmienić próg w już istniejącej regule alarmowania.

```

1 @mcp.tool()
2 def create_or_update_cpu_alert(
3     threshold: float,
4     alert_name: str = "AppHighCPUUsage",
5     namespace: str = "boutique-app"
6 ) -> dict:
7     """
8     Creates or updates Prometheus CPU usage alert for boutique app.
9     If it already exists it updates a threshold.
10
11     threshold is expressed in CPU cores.
12
13     Example:
14     - 0.05
15     - 0.1
16     """
17
18     yaml_manifest = f"""
19     apiVersion: monitoring.coreos.com/v1
20     kind: PrometheusRule
21
22     metadata:
23         name: "app-cpu-alert"

```

```

24     namespace: monitoring
25     labels:
26         release: monitoring
27
28     spec:
29         groups:
30         - name: app-cpu-alerts
31
32         rules:
33         - alert: {alert_name}
34           expr: |
35             sum(
36                 rate(
37                     container_cpu_usage_seconds_total{{
38                         namespace="{namespace}"
39                     }}[5m]
40                 )
41             ) > {threshold}
42           for: 30s
43
44         labels:
45             severity: warning
46
47         annotations:
48             summary: "High CPU usage in {namespace}"
49             description: "CPU usage exceeded threshold."
50     """
51
52     result = subprocess.run(
53         ["kubectl", "apply", "-f", "-"],
54         input=yaml_manifest,
55         text=True,
56         capture_output=True
57     )
58
59     return {
60         "success": result.returncode == 0,
61         "stdout": result.stdout,
62         "stderr": result.stderr
63     }

```

Definicja wszystkich komponentów niezbędnych do działania serwera MCP została zawarta w pliku `src/prometheus-mcp-server.yaml`. Wdrożenie konfiguracji realizowane jest za pomocą następującej komendy:

```
kubectl apply -f src/prometheus-mcp-server.yaml
```

W celu umożliwienia prawidłowego działania demonstracji wykonano przekierowanie portów (port-forwarding):

```
kubectl port-forward svc/prometheus-mcp-server 8000:8000 -n mcp-demo
```

Przekierowanie portów było niezbędne do umożliwienia komunikacji pomiędzy interfejsem webowym agenta AI a serwerem MCP działającym w klastrze Kubernetes.

7.4. Large Language Model

W projekcie wykorzystano model uruchamiany w środowisku **Ollama**, który był testowany podczas zajęć laboratoryjnych. W ramach implementacji przygotowano wdrożenie modelu wraz z interfejsem webowym umożliwiającym jego wykorzystanie.

W pierwszym kroku uruchomiono backend modelu Ollama, a następnie interfejs użytkownika typu web UI. Proces deploymentu został zrealizowany za pomocą manifestów Kubernetes:

```
kubectl apply -f src/ollama.yaml
kubectl apply -f src/ollama-web-ui.yaml
```

Po wdrożeniu komponentów skonfigurowano przekierowanie portów umożliwiające lokalny dostęp do usług działających w klastrze:

```
kubectl -n mcp-demo port-forward svc/open-webui 8080:8080
kubectl -n mcp-demo port-forward svc/ollama 11434:11434
```

Następnie pobrano model qwen3:0.6b do instancji Ollama za pomocą interfejsu API:

```
curl http://localhost:11434/api/pull \
-d '{"name": "qwen3:0.6b"}'
```

Poprawność procesu instalacji zweryfikowano poprzez zalogowanie do interfejsu Open WebUI dostępnego pod adresem `http://localhost:8080`. Po wybraniu pobranego modelu możliwe było wykonywanie zapytań i generowanie odpowiedzi, co potwierdziło poprawne działanie wdrożonego środowiska LLM.

Rozdział 8

Wdrożenie wersji demonstracyjnej

Demonstracja działania serwera MCP oraz agenta AI obejmuje scenariusz prezentujący możliwość zarządzania konfiguracją systemu monitoringu za pomocą języka naturalnego. W kolejnych etapach przedstawiono proces integracji serwera MCP z modelem językowym oraz przykłady operacji wykonywanych przez agenta AI na zasobach klastra Kubernetes.

8.1. Integracja serwera MCP z LLMem

Przygotowano plik konfiguracyjny `prometheus-mcp-description.json`, zawierający definicję połączenia z serwerem MCP.

```
1 [
2   {
3     "type": "openapi",
4     "url": "http://prometheus-mcp-server:8000",
5     "spec_type": "url",
6     "spec": "",
7     "path": "/openapi.json",
8     "auth_type": "none",
9     "key": "",
10    "info": {
11      "id": "prometheus-mcp-server",
12      "name": "prometheus-mcp-server",
13      "description": "Kubernetes MCP server for managing Prometheus
14      Operator configuration."
15    }
16  }
```

Następnie, analogicznie do ćwiczeń laboratoryjnych, zaimportowano przygotowaną konfigurację w interfejsie Open WebUI. W tym celu wybrano kolejno:

Admin Panel → Settings → Integrations → Add Connection

Po wybraniu opcji dodania nowego połączenia zaimportowano wcześniej przygotowany plik `prometheus-mcp-description.json`. Poprawność konfiguracji zweryfikowano za pomocą przycisku `Verify Connection`.

Po zapisaniu konfiguracji utworzono nowe okno chatu, a następnie aktywowano dodane narzędzie w interfejsie czatu.

8.2. Zmiana interwału pobierania metryk

W ramach demonstracji działania serwera MCP oraz agenta AI wykonano operację zmiany interwału pobierania metryk dla wybranego komponentu ServiceMonitor w klastrze Kubernetes.

W interfejsie czatu przekazano następujące polecenie:

Use the tool to set the scrape interval of ServiceMonitor "monitoring-prometheus-node-exporter" in namespace "monitoring" to 17 seconds

8.3. Dodanie nowej reguły alertowania

W celu zwizualizowania efektów drugiej funkcjonalności aplikacji, do agenta AI wyposażonego w to samo narzędzie skierowano poniższy prompt:

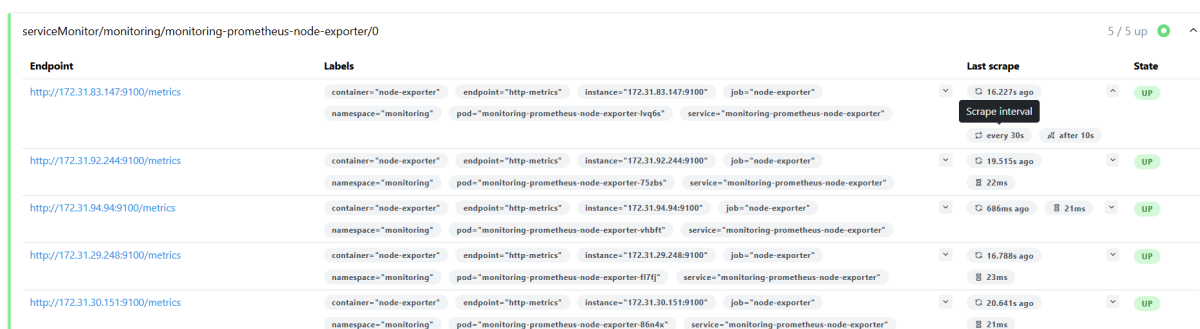
Use available tool to create alert about high cpu usage for app with threshold 0.05

Rozdział 9

Opis demonstracji

9.1. Zmiana interwału zbierania metryk

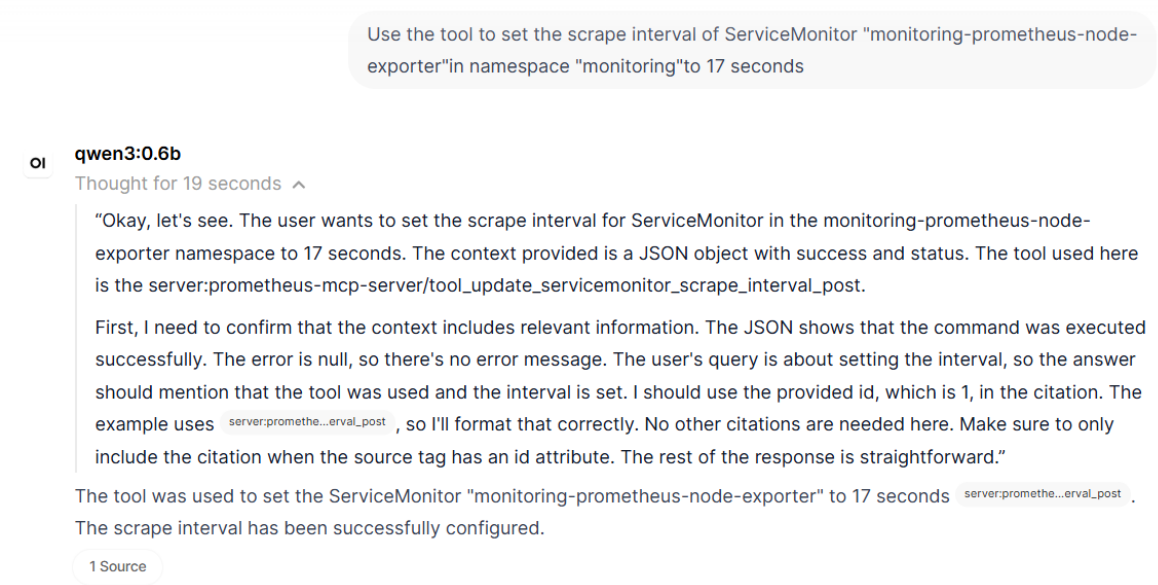
Przed wykonaniem kroków demonstracyjnych opisanych w rozdziale 8 możliwa jest weryfikacja aktualnej konfiguracji interwału w interfejsie Prometheus UI. W sekcji /targets widoczna jest domyślna wartość interwału zbierania metryk wynosząca 30 sekund (Rysunek 9.1).



Endpoint	Labels	Last scrape	State
http://172.31.83.147:9100/metrics	container="node-exporter" endpoint="http-metrics" instance="172.31.83.147:9100" job="node-exporter" namespace="monitoring" pod="monitoring-prometheus-node-exporter-lvq6s" service="monitoring-prometheus-node-exporter"	16:227s ago Scrape interval every 30s after 10s	UP
http://172.31.92.244:9100/metrics	container="node-exporter" endpoint="http-metrics" instance="172.31.92.244:9100" job="node-exporter" namespace="monitoring" pod="monitoring-prometheus-node-exporter-75zbs" service="monitoring-prometheus-node-exporter"	19:515s ago 22ms	UP
http://172.31.94.94:9100/metrics	container="node-exporter" endpoint="http-metrics" instance="172.31.94.94:9100" job="node-exporter" namespace="monitoring" pod="monitoring-prometheus-node-exporter-vhbft" service="monitoring-prometheus-node-exporter"	686ms ago 21ms	UP
http://172.31.29.248:9100/metrics	container="node-exporter" endpoint="http-metrics" instance="172.31.29.248:9100" job="node-exporter" namespace="monitoring" pod="monitoring-prometheus-node-exporter-f7fj" service="monitoring-prometheus-node-exporter"	16:788s ago 23ms	UP
http://172.31.30.151:9100/metrics	container="node-exporter" endpoint="http-metrics" instance="172.31.30.151:9100" job="node-exporter" namespace="monitoring" pod="monitoring-prometheus-node-exporter-86n4s" service="monitoring-prometheus-node-exporter"	20:641s ago 21ms	UP

Rysunek 9.1: Wstępna konfiguracja ServiceMonitora

Następnie w interfejsie czatu modelu językowego przekazano polecenie inicjujące zmianę konfiguracji interwału zbierania metryk w klastrze Kubernetes. Polecenie zostało przetworzone przez agenta AI, który wykorzystał serwer MCP do modyfikacji konfiguracji Prometheusa (Rysunek 9.2).



Rysunek 9.2: Polecenie oraz odpowiedź modelu AI

Po wykonaniu operacji oraz propagacji zmian w klastrze Kubernetes, w interfejsie Prometheus UI zaobserwowano aktualizację wartości interwału zbierania metryk (Rysunek 9.3).

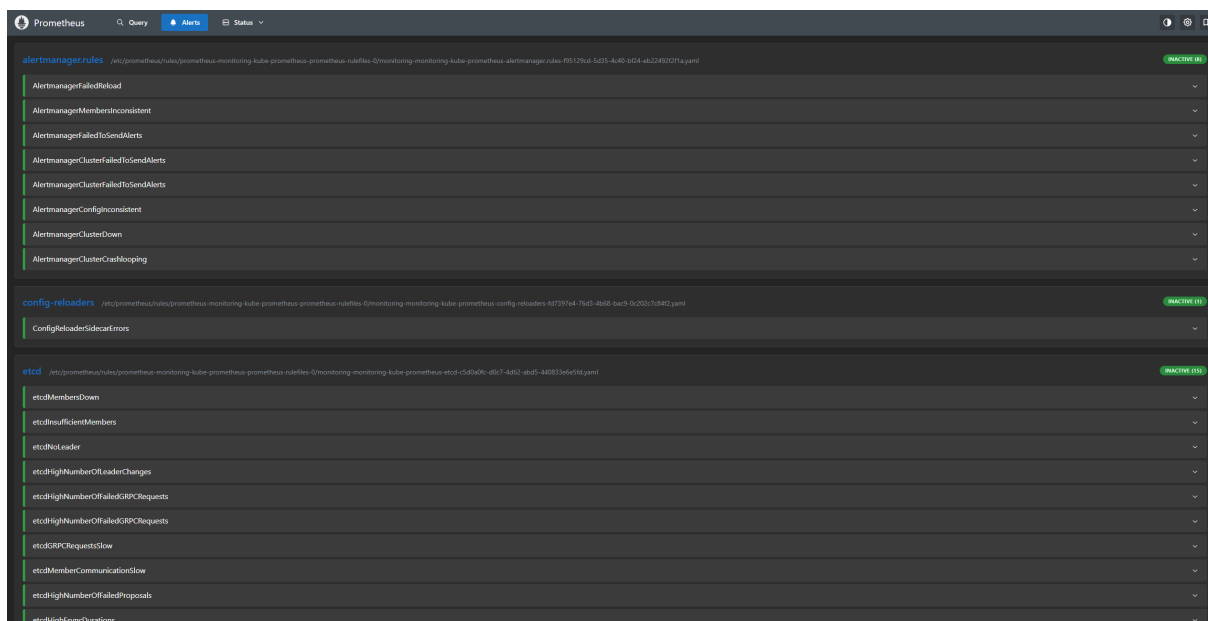
serviceMonitor/monitoring/monitoring-prometheus-node-exporter/0 5 / 5 up ^

Endpoint	Labels	Last scrape	State
http://172.31.30.151:9100/metrics	<code>container="node-exporter"</code> <code>endpoint="http-metrics"</code> <code>instance="172.31.30.151:9100"</code> <code>job="node-exporter"</code> <code>namespace="monitoring"</code> <code>pod="monitoring-prometheus-node-exporter-86n4x"</code> <code>service="monitoring-prometheus-node-exporter"</code>	10.554s ago Scrape interval every 17s after 10s	UP
http://172.31.83.147:9100/metrics	<code>container="node-exporter"</code> <code>endpoint="http-metrics"</code> <code>instance="172.31.83.147:9100"</code> <code>job="node-exporter"</code> <code>namespace="monitoring"</code> <code>pod="monitoring-prometheus-node-exporter-lvq6s"</code> <code>service="monitoring-prometheus-node-exporter"</code>	29.296s ago 29ms every 17s after 10s	UP
http://172.31.92.244:9100/metrics	<code>container="node-exporter"</code> <code>endpoint="http-metrics"</code> <code>instance="172.31.92.244:9100"</code> <code>job="node-exporter"</code> <code>namespace="monitoring"</code> <code>pod="monitoring-prometheus-node-exporter-75zbs"</code> <code>service="monitoring-prometheus-node-exporter"</code>	27.509s ago 23ms every 17s after 10s	UP
http://172.31.94.94:9100/metrics	<code>container="node-exporter"</code> <code>endpoint="http-metrics"</code> <code>instance="172.31.94.94:9100"</code> <code>job="node-exporter"</code> <code>namespace="monitoring"</code> <code>pod="monitoring-prometheus-node-exporter-vh8ft"</code> <code>service="monitoring-prometheus-node-exporter"</code>	19.693s ago 39ms every 17s after 10s	UP
http://172.31.29.248:9100/metrics	<code>container="node-exporter"</code> <code>endpoint="http-metrics"</code> <code>instance="172.31.29.248:9100"</code> <code>job="node-exporter"</code> <code>namespace="monitoring"</code> <code>pod="monitoring-prometheus-node-exporter-ftfj"</code> <code>service="monitoring-prometheus-node-exporter"</code>	19.907s ago 20ms every 17s after 10s	UP

Rysunek 9.3: Konfiguracja ServiceMonitora po wykonaniu zapytania

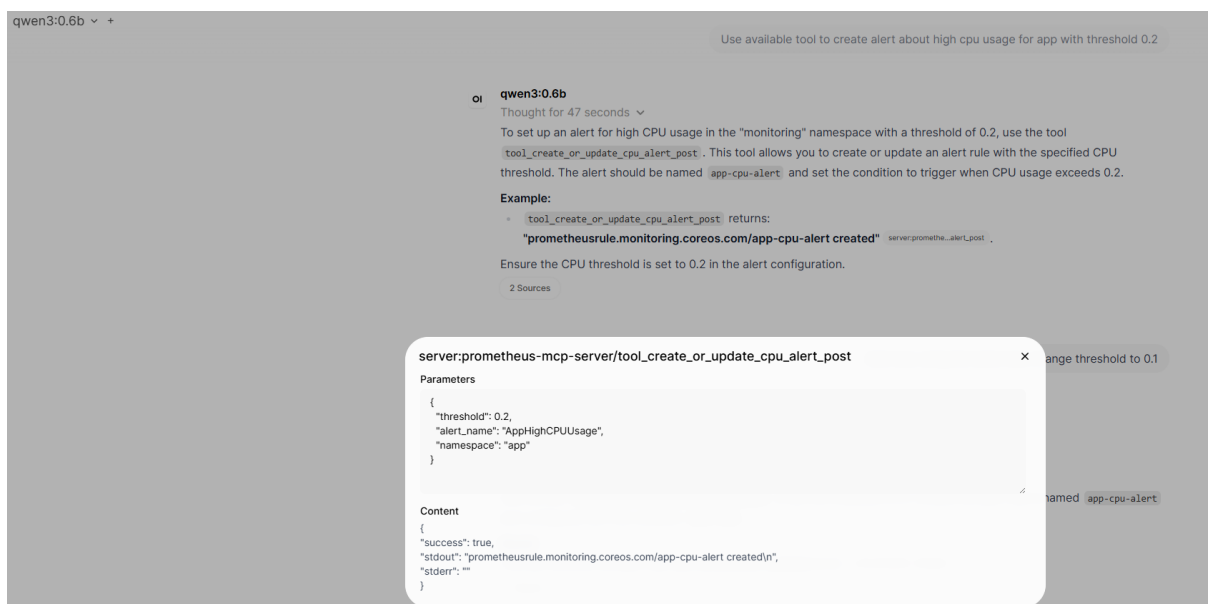
9.2. Stworzenie reguły alertowania

Przed wysłaniem promptu do agenta, na naszej lokalnej maszynie, na porcie z przekierowanym systemem monitoringu Prometheus w zakładce Alerts, możemy znaleźć grupy i należące do poszczególnych reguły alarmowania. Wszystkie przedstawione są domyślnie utworzone przez wykonaną konfigurację. Wśród nich nie ma żadnego systemu ostrzegania związanego z obserwacją zasobów wykorzystanych przez aplikację sklepu internetowego (Rysunek 9.4).



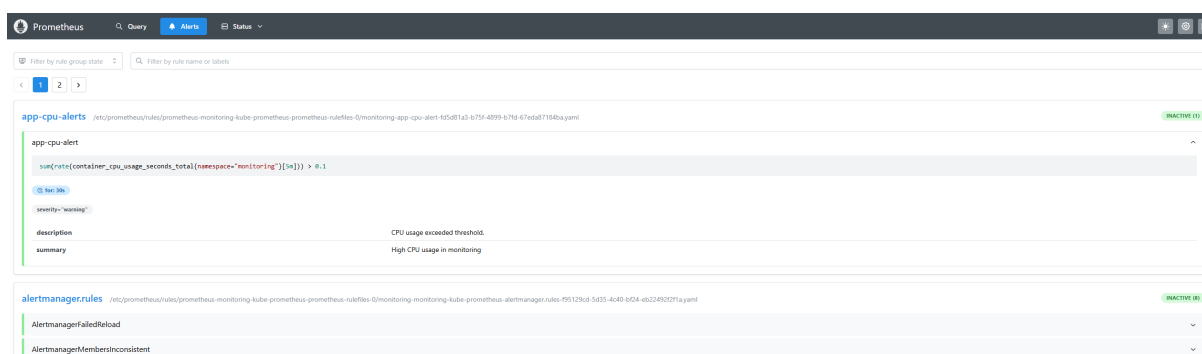
Rysunek 9.4: Stan sprzed dodania reguły

Skierowano się więc do open-web-ui i konwersacji z agentem AI. Przy pomocy wspomnianego powyżej prompta uzyskano zmianę konfiguracji, a w efekcie także i odpowiedź na zapytanie (Rysunek 9.4).



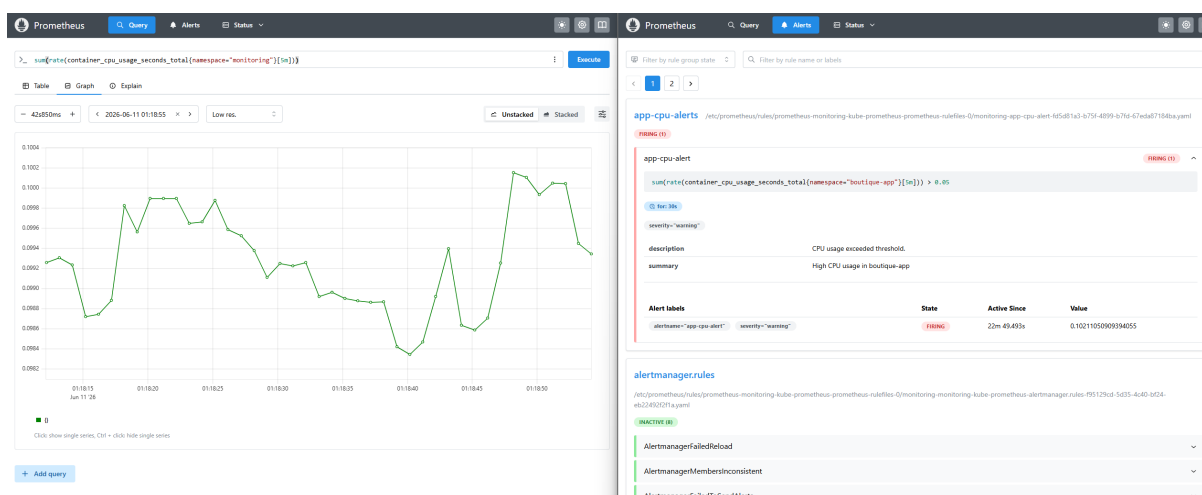
Rysunek 9.5: Konwersacja z LLM-em

Poprawnie przeprowadzoną konfigurację można było zauważyć zarówno z odpowiedzi zapytania (Rysunek 9.5), jak i z samej strony alertów, która po odświeżeniu na samej górze wskazała nową grupę alertów - app-cpu-alerts (Rysunek 9.6).



Rysunek 9.6: Stan reguł po wykonaniu prompta

Z obserwacji można było zauważyć, że aktualny status reguły to Inactive. Reguła odczytywała i śledziła zasób, ale poziom zużycia był na tyle niski, że nie przekraczał on wskazanego progu. W celu zaobserwowania działania reguły postanowiono zadać kolejny prompt zmniejszający próg tolerancji na taki, który powinien wskazać stan alarmowy. Aby wyznaczyć ten próg posłużono się narzędziem pozwalającym na zadanie zapytania PromQL w Prometheus o treści dokładnie takiej samej, jaka jest w warunku reguły alertowania. W tym przypadku, po rekonfiguracji reguły, jej stan zmienił się na Firing (Rysunek 9.7).



Rysunek 9.7: Zaktualizowana reguła. Nowo wyznaczony próg to 0.05 zużycia jednego rdzenia procesora

Rozdział 10

Podsumowanie i wnioski

Celem projektu było rozszerzenie klasycznego systemu obserwowalności o warstwę umożliwiającą jego programowalne sterowanie z wykorzystaniem dużego modelu językowego oraz serwera MCP.

Najważniejszym osiągnięciem pracy było skuteczne zintegrowanie LLM z infrastrukturą Prometheusa poprzez MCP Server, który pełnił rolę warstwy pośredniej tłumaczącej polecenia w języku naturalnym na operacje konfiguracyjne w klastrze Kubernetes.

W ramach demonstracji działania systemu udało się:

- zmodyfikować interwał zbierania metryk (scrape interval) dla wybranego ServiceMonitora,
- utworzyć nową regułę alertowania dotyczącą wysokiego użycia CPU przez aplikację,
- zweryfikować propagację zmian w środowisku Prometheus oraz ich wpływ na działanie systemu.

Przeprowadzone testy potwierdziły, że integracja MCP i LLM pozwala na skuteczne zarządzanie konfiguracją systemu obserwowalności bez konieczności ręcznej edycji manifestów Kubernetes, upraszczając tym samym proces administracji i zwiększając jego elastyczność.